Comparing an MLP and a CNN for Connect Four using the AlphaZero approach

Julian Albers
Deep Neural Networks, ELE680
https://github.com/julian-al-03/Connect-Four-RL
November 6, 2026

1 Introduction

In the following we describe how the AlphaZero approach is used to build agents that play Connect4. In particular we compare an MLP and CNN of comparable size. Furthermore we answer two questions:

1. Does the CNN outperform the MLP when being trained in the same pipeline?
2. How much do the `simulations`, `temperature_moves` and `c_puct` settings matter?

In the following we give an overview of the AlphaZero approach, as well as our implementation of it. Afterwards we describe the experiments and interpret their results in order to answer the proposed questions.

2 Methods and Materials

2.1 The General Approach

We implement the code from scratch with the help of AI (Claude). For training our networks we use PyTorch. For the AlphaZero pipeline specifically we do not use any third party implementation directly. Although they exist (for example https://github.com/suragnair/alpha-zero-general), we choose to do the implementation from scratch for two reasons; first because it gives us total control over the code, second the gradual development of the code gives a deeper understanding of the approach.

2.2 The AlphaZero Approach

In the following we give an overview of the AlphaZero approach described by Silver et al. [1]. AlphaZero is an iterative algorithm that consists of two major steps. First we use our network to generate training data, using Monte Carlo tree search (MCTS). Then we use this training data to improve our network. At least in theory this should give us a loop of: improving network $\rightarrow$ improving training data $\rightarrow$ improved network $\rightarrow \dots$. Our network is designed to give us a number for each move that we can make (In the case of Connect4 this is 7, since there are 7 columns) as well as a prediction if we win (1), lose (-1) or draw (0). We refer to the parts of the network as the policy head and the value head respectively.

2.3 Our Implementation

Building the MCTS: We have three hyperparameters to control how the MCTS tree is built. First is the **`simulations`** hyperparameter. This controls the number of simulations we make to build the MCTS. Descending down the MCTS and reaching a non expanded (or terminal) node is counted as one simulation. Second is the Dirichlet noise. As in AlphaGo Zero [2], we add Dirichlet noise to the priors at the root. This noise is controlled by the hyperparameter **`dirichlet_alpha`**. The noise is used to encourage also the use of moves the network currently dislikes. Lastly we also use the PUCT score [2] which controls the tradeoff between exploration and exploitation. The hyperparameter is **`c_puct`**, where a higher `c_puct` corresponds to more exploration and a smaller `c_puct` to less exploration.

Generating Training Data: The network generates its own training data by playing games against itself (self-play). For each position s in a game, we run MCTS from s and normalize the visit counts at the root into a distribution $\boldsymbol{\pi}$:

$$\boldsymbol{\pi}_a = \frac{N(s,a)}{\sum_b N(s,b)} \tag{1}$$

Here $N(s,a)$ is the visit count for move a from state s. Together with the final outcome of the game z, we generate a training sample $(s, \boldsymbol{\pi}, z)$. To make games differ, we introduce a hyperparameter **`temperature_moves`**. For the first `temperature_moves` moves of the game, the moves are sampled from $\boldsymbol{\pi}$, after that, the most visited move is played. New training examples are added to a buffer, where the **`buffer_size`** is also a hyperparameter, which controls the buffer size. The oldest training data gets dropped. Furthermore, we use the trick mentioned in [2], to get more training data by mirroring our data. Since Connect4 is symmetric, we can just mirror our board and distribution $\boldsymbol{\pi}$ to get another valid example.

Training the Network: We have three ways of controlling how the AlphaZero algorithm is used to train our network. First we have the **`iterations`** hyperparameter, that controls how many iterations of the AlphaZero algorithm we make. Next we have **`games_per_iteration`** which controls how many self-play games we play per iteration. This controls indirectly how many new training data we generate per iteration. The last two hyperparameters **`train_steps`** and **`batch_size`** control how much the network is trained each step. In each iteration we train the network on `train_steps` batches of size `batch_size` drawn from the buffer.

As an error function we use the loss described in [1]. For a sample $(s, \boldsymbol{\pi}, z)$ with network output $(\boldsymbol{p}, v)$, we minimize

$$L = (z - v)^2 - \boldsymbol{\pi}^\top \log \boldsymbol{p} \tag{2}$$

The first term trains the value head to predict the game outcome, and the second trains the policy head to match the search distribution. We use the Adam optimizer with learning rate **`lr`** and L2 regularization through **`weight_decay`**.

All the hyperparameters are summarized in Table 1.

Network Architectures: We compare two networks with almost the same number of parameters (MLP: 171,656; CNN: 171,178), to make sure that the difference in strength is not just due to one network being larger. Both receive the board as two 6×7 binary planes (84 bit input). One plane with the pieces of the player whose turn it is, and one with the opponent's pieces. Illegal moves are masked out of the policy output, and the value output passes through a tanh such that it is in $[-1, 1]$.

The **MLP** flattens the board into 84 inputs, followed by three fully connected layers of size 256 with ReLU activations. The policy head is a single linear layer with 7 outputs. The value head has one additional hidden layer of size 64 and one output.

The **CNN** is a smaller version of the residual architecture of AlphaGo Zero [2], using the same residual blocks of He et al. [3]. An initial 3×3 convolution with 48 channels is followed by four residual blocks. Each block consists of two 3×3 convolutions with batch normalization and a skip connection. The policy head reduces the features to 2 channels with a 1×1 convolution, followed by a linear layer with 7 outputs. The value head reduces them to 1 channel, followed by a hidden layer of 64 units and one output.

The idea is that the CNN has an advantage over the MLP since it can use adjacency information: Because the input is flattened, the MLP does not know which cells are adjacent, and it has to learn

similar patterns multiple times, depending on where they appear on the board. Convolutions use the same weights across the board, a pattern learned in one place is recognized everywhere, which is useful in Connect4, where the same patterns occur at many positions.

Hyperparameter	Value	Swept values
`simulations`	100	25, 50, 100, 200
`temperature_moves`	10	0, 5, 10, 20
`c_puct`	1.5	0.5, 1.5, 4
`dirichlet_alpha`	1.0	–
`iterations`	100	–
`games_per_iteration`	100	–
`train_steps`	400	–
`batch_size`	128	–
`buffer_size`	100,000	–
`lr`	0.001	–
`weight_decay`	0.0001	–

Table 1: Hyperparameters of the main runs. Sweeps were run for both MLP and CNN and changed one hyperparameter at a time.

2.4 Experiments

All our experiments use the pipeline with the hyperparameters as in Table 1, unless stated otherwise. In order to make our results more robust we carry out all experiments with three different random seeds, making the results reproducible.

We first compare the MLP and the CNN under identical settings. We then study the effect of individual hyperparameters by changing one at a time, as described by the swept values in Table 1.

Finally, to test whether reusing samples limits the playing strength, we train three additional MLP runs with 500 games per iteration that empty the buffer at the start of every iteration, so they train only on the newest games.

2.5 Evaluation Approach

Agents: Unless stated otherwise, agents select moves by running MCTS with 100 simulations, and playing the most visited move. Here we do not use Dirichlet noise or temperature. All agents, including the ones trained with different values in a sweep, are evaluated with these search settings, so that differences in strength come from training. We also evaluate the agent's network *raw*, meaning without any search, just using the priors from the network directly.

Opponents: We test the MLP and CNN agents against each other and also against three baseline agents, ordered in strength:

1. *Random Agent* plays uniformly random moves.
2. *Greedy Agent* checks for winning move, blocks an immediate win of the opponent and plays random otherwise.
3. *Pure MCTS* n uses MCTS with uniformly random priors and n simulations to choose a move.

Games: Without any noise or temperature the MLP and CNN agents become deterministic, therefore two agents would always repeat the same game against each other. Therefore, we always start each game with an opening of four random moves. Since the random opening could favor one side, both agents play both colors of each opening.

Metrics: We report the score of an agent over N games with W wins and D draws:

$$S = \frac{W + D/2}{N} \tag{3}$$

A score of 0.5 means that both agents are equally strong under these conditions. We use $N = 200$ games to compare two agents.

3 Results

MLP vs CNN, direct comparison: We use the values for the hyperparameters in Table 1 to train the MLP and CNN agent. We then evaluate them against our baseline models, the results are found in Table 2. The CNN outperforms the MLP against all opponents. We also compare the CNN against the MLP directly. With search the CNN scores 0.655 (0.590–0.715) against the MLP and 0.708 (0.640–0.750) without search, averaged over the three seeds with the lowest and highest scoring seed in brackets. While the networks are comparable in size, the CNN's advantage still comes at the cost of training time. On a shared machine the MLP took 1.16h for 100 training iterations, while the CNN took 4h for 100 training iterations. The difference is most visible in the raw comparison, suggesting that a lot of the weaknesses of the MLP network are hidden by the MCTS search.

Agent	Random	Greedy	Pure MCTS 100	Pure MCTS 400
MLP + search	0.997 (0.995–1.000)	0.929 (0.907–0.945)	0.753 (0.695–0.818)	0.522 (0.505–0.547)
CNN + search	0.999 (0.998–1.000)	0.966 (0.960–0.973)	0.851 (0.802–0.885)	0.700 (0.682–0.725)
MLP raw	0.884 (0.863–0.905)	0.498 (0.450–0.555)	0.277 (0.233–0.345)	0.168 (0.120–0.198)
CNN raw	0.983 (0.980–0.988)	0.861 (0.853–0.875)	0.630 (0.578–0.677)	0.448 (0.422–0.490)

Table 2: Scores of the MLP and CNN networks after 100 training iterations. We play 200 games per pairing. Each cell shows the mean over three seeds, with the lowest and highest seed in brackets. "Search" uses MCTS with 100 simulations per move, and "raw" plays the move with the highest prior without search.

Parameter sweeps: As a second experiment we perform several parameter sweeps for both the MLP and CNN. The results are shown in Table 3 and Table 4. Note that the scores are performed under the same conditions as the scores in Table 2, in particular we use 100 simulations during evaluation, regardless of the values we used for the `simulations` parameter during training. First we sweep `temperature_moves` and `c_puct`, which both play into the notion of exploration vs exploitation. For both networks less exploration (`temperature_moves` [0,5], `c_puct` 0.5), results in worse performance, although the effect is more drastic for the MLP. Too much exploration (`temperature_moves` 20, `c_puct` 4) results in more comparable results for both the MLP and CNN.

We also sweep the `simulations` parameter. Changing the number of simulations changed the scores by at most about 0.06, and most seed ranges overlap with those of the main runs. The only clear difference is the MLP trained with 25 simulations, which scores lower against pure MCTS 400.

Training dynamics: The scores against pure MCTS stop improving after about 30 iterations (Figure 2), while the training losses keep decreasing (Figure 1) and every checkpoint still beats its predecessor (Figure 3). A decreasing training loss therefore does not translate into a stronger agent.

The value loss rises during the first 20 to 25 iterations and then starts dropping (Figure 1). One possible explanation is that early self-play games are easier to predict, since the agent is playing worse, and simple mistakes can decide the game.

Training only on new games: The fact that the policy loss keeps decreasing (Figure 1) while the performance against pure MCTS and the greedy opponents stagnates might be interpreted as the agents overfitting to the training data. In order to test this we run another experiment with the

MLP network. We use the settings from Table 1, and set `games_per_iteration` to 500 instead of 100. Furthermore, we use an empty buffer in each new iteration. The values for the policy and value loss are shown in Figure 4 and the scores against pure MCTS and greedy are shown in Figure 5.

During training, these runs score slightly higher against pure MCTS than the main runs from iteration 40 on, but within the range of the seeds (Figure 5). Their policy loss ends at about the same level as in the main runs, while their value loss stays clearly higher, at about 0.43 instead of 0.33 (Figure 4).

Hyperparameter	Value	vs. pure MCTS 100	vs. pure MCTS 400
`temperature_moves`	0	0.508 (0.470–0.530)	0.328 (0.278–0.360)
	5	0.650 (0.593–0.690)	0.464 (0.400–0.522)
	10	0.753 (0.695–0.818)	0.522 (0.505–0.547)
	20	0.725 (0.710–0.745)	0.492 (0.482–0.502)
`c_puct`	0.5	0.633 (0.588–0.680)	0.473 (0.445–0.525)
	1.5	0.753 (0.695–0.818)	0.522 (0.505–0.547)
	4	0.730 (0.700–0.755)	0.524 (0.485–0.555)
`simulations`	25	0.690 (0.675–0.718)	0.466 (0.435–0.485)
	50	0.742 (0.698–0.767)	0.481 (0.460–0.507)
	100	0.753 (0.695–0.818)	0.522 (0.505–0.547)
	200	0.714 (0.690–0.738)	0.471 (0.412–0.515)

Table 3: Final scores of the MLP hyperparameter sweeps, shown as the mean over three seeds with the lowest and highest seed in brackets. Each network played 200 games per opponent. Values in bold are the same MLP runs shown in Table 2. Swept values apply to training only: all agents were evaluated with 100 simulations per move and `c_puct` = 1.5.

Hyperparameter	Value	vs. pure MCTS 100	vs. pure MCTS 400
`temperature_moves`	0	0.736 (0.728–0.743)	0.492 (0.448–0.550)
	5	0.827 (0.805–0.870)	0.620 (0.595–0.637)
	10	0.851 (0.802–0.885)	0.700 (0.682–0.725)
	20	0.853 (0.825–0.882)	0.692 (0.672–0.723)
`c_puct`	0.5	0.759 (0.723–0.780)	0.623 (0.600–0.637)
	1.5	0.851 (0.802–0.885)	0.700 (0.682–0.725)
	4	0.823 (0.767–0.875)	0.669 (0.640–0.700)
`simulations`	25	0.825 (0.797–0.863)	0.635 (0.603–0.688)
	50	0.829 (0.797–0.848)	0.657 (0.613–0.698)
	100	0.851 (0.802–0.885)	0.700 (0.682–0.725)
	200	0.848 (0.818–0.865)	0.690 (0.647–0.718)

Table 4: Final scores of the CNN hyperparameter sweeps, shown as the mean over three seeds with the lowest and highest seed in brackets. Each network played 200 games per opponent. Values in bold are the main CNN runs, so these rows contain the same results. Swept values apply to training only: all agents were evaluated with 100 simulations per move and `c_puct` = 1.5.

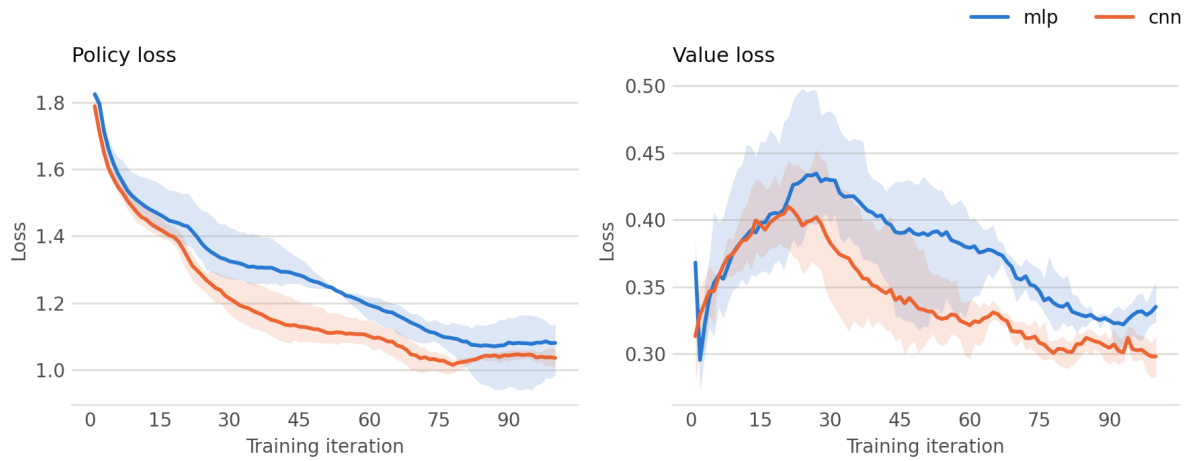


Figure 1: Policy loss (cross-entropy between the search distribution $\boldsymbol{\pi}$ and the network's policy) and value loss (squared error between the game outcome z and the predicted value v) during training. Lines show the mean over three seeds, shaded bands the range from lowest to highest seed.

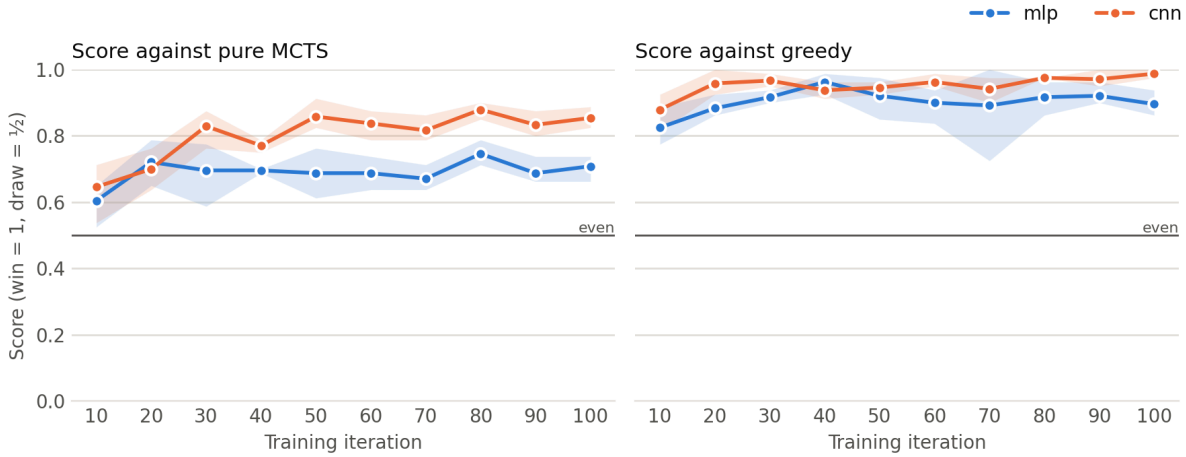


Figure 2: Score during training against pure MCTS with 100 simulations (left) and the greedy agent (right). Every 10 iterations, each network plays 40 games against each opponent, selecting moves by MCTS with 100 simulations. Lines show the mean over three seeds, shaded bands the range from lowest to highest seed.

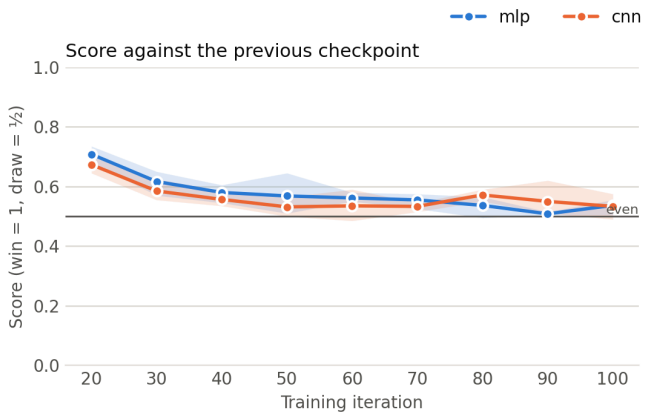


Figure 3: Score of each checkpoint against the checkpoint 10 iterations earlier, with 200 games per evaluation, played with 100 simulations per move. Lines show the mean over three seeds, shaded bands the range from lowest to highest seed.

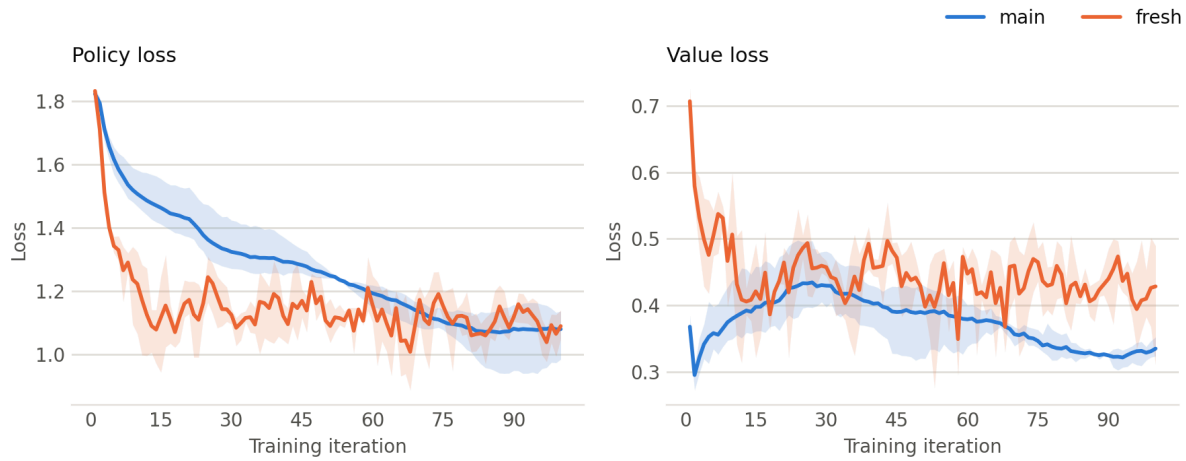


Figure 4: Policy and value loss of the main MLP runs and the runs trained only on the newest games ("fresh", 500 games per iteration). Each point is the average over the 400 gradient steps of one iteration. In the fresh runs, each sample is used only about 2 to 3 times, compared with about 10 times in the main runs, so their losses are measured on less familiar data and are not directly comparable. Lines show the mean over three seeds, shaded bands the range from lowest to highest seed.

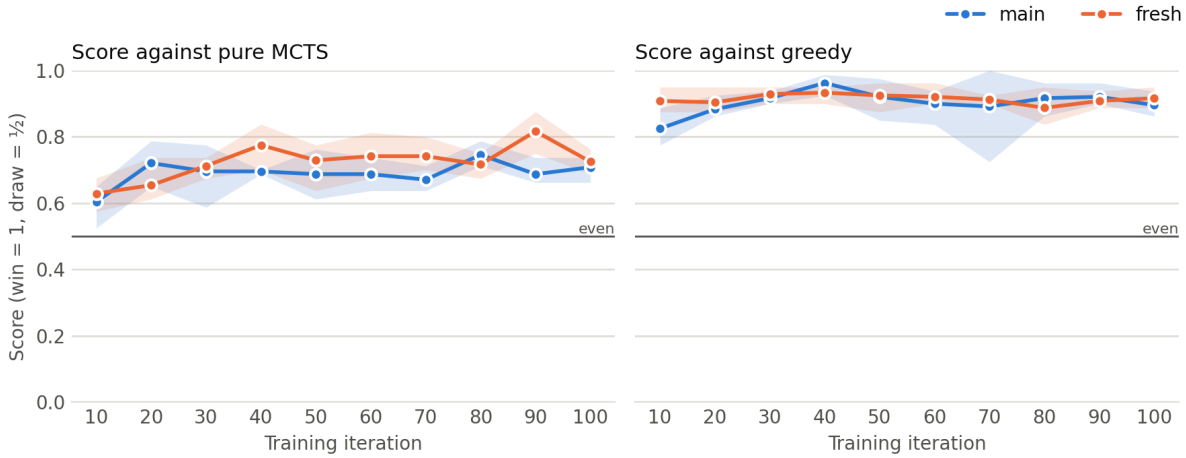


Figure 5: Score of the MLP during training against pure MCTS with 100 simulations (left) and the greedy agent (right). "Main" are the main runs with 100 games per iteration and a buffer of 100,000 samples. "Fresh" are runs with 500 games per iteration that train only on the games of the current iteration. Every 10 iterations, each network plays 40 games per opponent with 100 simulations per move. Lines show the mean over three seeds, shaded bands the range from lowest to highest seed.

4 Discussion

1. Does the CNN outperform the MLP when being trained in the same pipeline?

Yes, the CNN does outperform the MLP clearly. It is reasonable to assume that the advantage comes from the shared weights of the CNN; it is able to recognize patterns, such as an 'open three' regardless where on the board it appears. The MLP has to learn the patterns separately for each location. This also matches the use of CNNs in AlphaGo Zero and AlphaZero [1], [2]. The gap between the networks is larger without search (0.708) than with search (0.655), suggesting that the search compensates for part of the MLP's weaker priors. Without search, the MLP is about as strong as the greedy agent (0.498), which only checks for immediate wins and blocks, while the raw CNN clearly beats it (0.861). With search, the MLP scores 0.929 against the greedy agent, so the search adds much of the MLP's strength.

The CNN needed about 3.5 times the training time. However, Figure 2 suggests that the CNN reaches the final level of the MLP after about 20-30 iterations, which takes about the same time as the full MLP run, so the CNN may be preferable even for a fixed training time. Note that this estimate is rough,

because the scores during training are based on only 40 games per seed and the timings were measured on a shared machine.

Finally, the CNN also uses batch normalization and skip connections, so its advantage cannot be credited exclusively to convolutions.

2. How much do the exploration and simulation settings matter?

The exploration vs exploitation tradeoff matters more than the amount of simulations. Reducing exploration lowered the score against pure MCTS 100 by up to 0.245 for the MLP (`temperature_moves` 0) and 0.115 for the CNN, and a smaller `c_puct` lowered it by about 0.1 for both networks. Changing the number of simulations changed the scores only by at most about 0.06.

The likely reason is the variance in the produced training data. Without sampled moves (`temperature_moves` 0), we will produce a lot of similar or equal games. A small `c_puct` additionally favors exploitation, which concentrates the search on the network's preferred moves, and the search can correct the network's mistakes less. Since our evaluation starts from random openings, networks trained on less varied games may also meet unfamiliar positions. The MLP suffers more from reduced exploration, which fits its lack of weight sharing: it cannot transfer what it learned in one position to similar positions and therefore depends more on varied data.

On the flip side, more exploration than in our main runs did not hurt. The problem with too much exploration is generally that the search is not able to discover good moves. However, the 100 simulations we use might be enough to find good moves even when rewarding exploration more.

It is surprising that increasing `simulations` did not help, since AlphaZero relies on search to improve its training targets. One explanation is that Connect4 only needs a few simulations, since there are at most 7 legal moves, so even 25 simulations can visit each move at the root several times. Additional simulations may change the search distribution only little. The slightly lower scores for 200 simulations lie within the variation between seeds, so they may be noise. If they are real, a possible reason is reduced exploration: with more simulations, the visit counts concentrate more on the best-looking moves. Sampling moves from this sharper distribution then produces less varied games, similar to a lower `temperature_moves`. With fewer simulations, in turn, the search targets are noisier. We did not measure how varied the games are, so this remains a hypothesis. With a fixed compute budget, fewer simulations may be a good trade-off between compute budget and performance.

3. Additional observations on training dynamics and overfitting

The runs that train only on new games (Figure 4, Figure 5) give a first test of the overfitting hypothesis. Their value loss, measured on the fresh data, is clearly higher than in the main runs, which suggests that the value head of the main runs partly fits samples it has seen many times. However, the final strength did not change, even though these runs used five times as many self-play games per iteration. Reusing samples therefore does not seem to be what limits the playing strength. The policy loss after 100 iterations is comparable in both settings, however it does drop faster in the beginning, when using only fresh data. One possible explanation can simply be that the 500 games per iteration instead of 100 give more positions early on, when the network is learning the most. It is left for future work to determine if the agents overfit their training data, and where the mismatch between decreasing policy loss (Figure 1) and stagnating performance (Figure 2) comes from.

4.1 Limitations

Evaluation: Our scores only measure strength relative to a few fixed opponents. The strongest of them is pure MCTS with 400 simulations. So our results do not show how close the agents are to optimal play.

Statistics: We trained every configuration with only three seeds, and the seed ranges of many sweep results overlap. Smaller effects can therefore not be distinguished from random variation.

Hyperparameters: Several hyperparameters were never tuned, including the learning rate, the number of gradient steps, the Dirichlet noise and the network sizes. Furthermore, the sweeps changed only one hyperparameter at a time, not taking into account how they might interact.

Timing: All training times are on a shared CPU machine with several runs in parallel, so they are only approximate.

4.2 Future Work

Absolute evaluation: Connect Four is solved [4], so an existing solver could label the optimal moves in a set of test positions. The fraction of optimal moves an agent plays would measure its strength directly instead of relative to sub-optimal baselines.

Overfitting and data reuse: Our runs that train only on new games give a first indication, but they change the number of games and the buffer at the same time. Varying these two separately, and measuring the loss on games that are never used for training, would show whether the networks overfit and why the policy loss keeps decreasing while the playing strength stagnates.

Testing the explanations: Measuring how varied the self-play games are, and how sharp the search targets become, would test our explanations for the effect of exploration. Training a CNN without batch normalization and skip connections would show how much of its advantage comes from convolutions alone.

Broader experiments: More seeds, longer training, and sweeps of the untuned hyperparameters and their combinations would make the results more reliable.

References

[1] D. Silver *et al.*, "A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play," *Science*, vol. 362, no. 6419, pp. 1140–1144, 2018, doi: 10.1126/science.aar6404.

[2] D. Silver *et al.*, "Mastering the game of Go without human knowledge," *Nature*, vol. 550, pp. 354–359, 2017, doi: 10.1038/nature24270.

[3] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[4] L. V. Allis, "A Knowledge-Based Approach of Connect-Four: The Game is Solved: White Wins," Master's thesis, 1988.